



PRAKTIKUM PIRANTI CERDAS

BUKU PANDUAN

TIM PENYUSUN:

ZAMAH SARI, S.T, M.T

IMELDA AZALIYA RAHMA

MUHAMMAD NIZAR ZULMI ROHMATULLOH

PRESENTED BY: LAB. INFORMATIKA

UNIVERSITAS MUHAMMADIYAH MALANG

DAFTAR ISI

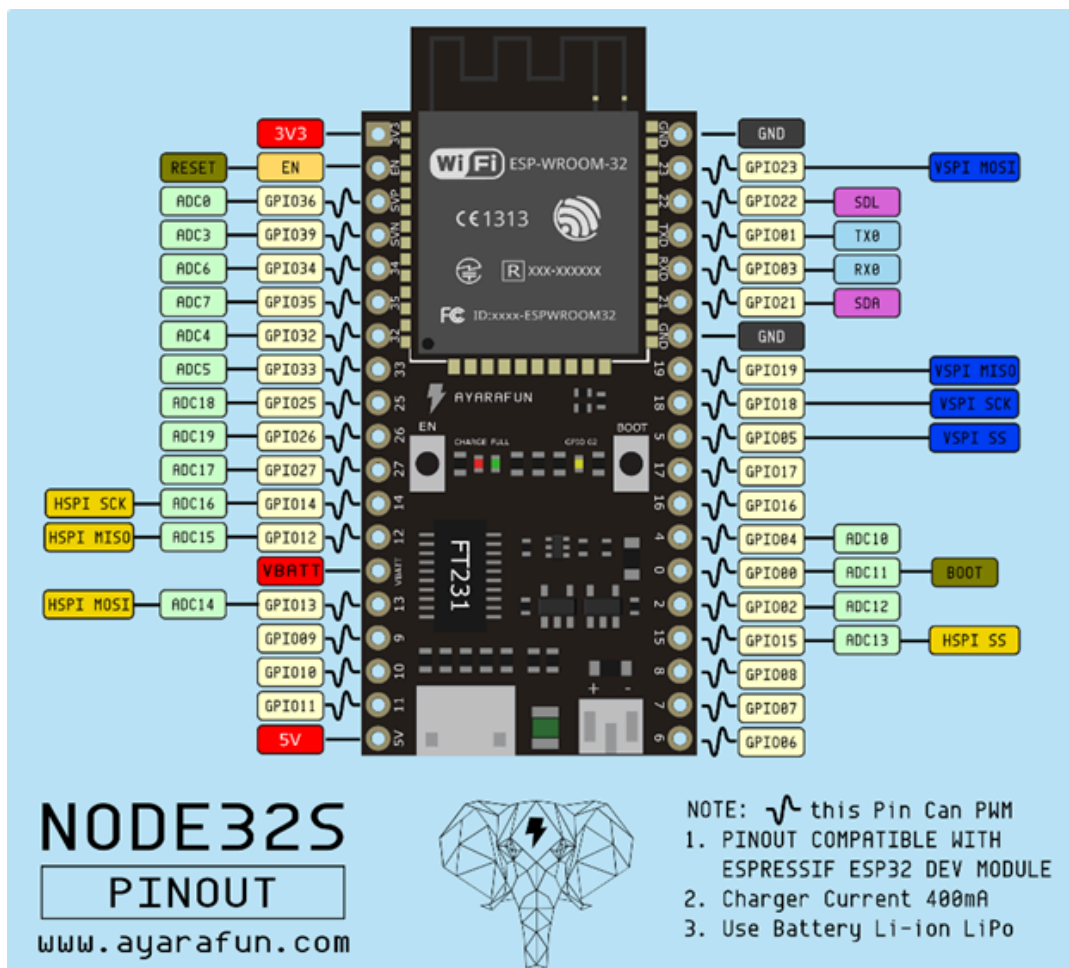
DAFTAR ISI.....	2
BAB 1: PENGENALAN MIKROKONTROLER & INPUT DAN OUTPUT.....	2
1. Mikrokontroler.....	3
o NodeMCU ESP32.....	3
2. Pin pada Mikrokontroler.....	4
o Pin Input.....	4
o Pin Output.....	4
3. Arduino IDE.....	5
4. Fungsi-Fungsi Dasar Pemrograman Mikrokontroler.....	5
5. Komponen: Switch.....	7
BAB 2: PENGENALAN DIGITAL DAN ANALOG INPUT DAN OUTPUT.....	14
1. Pengenalan Digital dan Analog Input dan Output.....	14
2. I/O Digital dan Analog.....	14
a. ADC (analog to digital Converter) dan DAC (Digital To Analog Converter).....	15
3. Perangkat Input Output.....	17
a. Perangkat Input.....	18
b. Perangkat Output.....	19
BAB 3: KONEKSI WIFI.....	22
1. WiFi pada ESP32.....	22
a. Menghubungkan ke Jaringan WiFi.....	22
b. Memeriksa Status Koneksi WiFi pada ESP32.....	23
2. Komponen: MPU6050.....	24
3. Komponen: Buzzer.....	24
BAB 4: PROTOKOL KOMUNIKASI.....	25
1. HTTP.....	25
2. Pengiriman Data Ke Database Dengan Protokol HTTP.....	25
a. Database.....	25
b. Pengiriman Data ke Database.....	26
3. MQTT.....	27
a. Arsitektur MQTT: Pub/Sub Pattern.....	27
b. Client dan Broker (Server).....	27
c. Topik.....	28
d. Quality of Service (QoS).....	29
e. Studi Kasus: Mencoba MQTT.....	30
BAB 5: INTERNET OF THINGS (IOT).....	38
1. Relay.....	38
2. Blynk.....	39

BAB 1: PENGENALAN MIKROKONTROLER & INPUT DAN OUTPUT

1. Mikrokontroler

Mikrokontroler adalah sebuah komponen dengan ukuran minimalis yang berfungsi sebagai pengendali sistem. Mikrokontroler merupakan sebuah sistem mikroprosesor lekap yang dikemas dalam sebuah chip. Mikrokontroler berbeda dengan mikroprosesor yang digunakan dalam sebuah PC. Di dalam Mikrokontroler terdapat sistem pendukung minimal seperti mikroprosesor, memori, I/O Interface, sedangkan dalam Mikroprosesor hanya berisi CPU saja. Mikrokontroler memiliki beberapa macam prototyping board yang berbeda-beda dengan tipe mikrokontroler yang berbeda pula.

○ NodeMCU ESP32



ESP32 adalah mikrokontroler yang dikenalkan oleh Espressif System dan merupakan penerus dari mikrokontroler ESP8266. Pada mikrokontroler ini sudah

tersedia modul wifi dalam chip sehingga sangat mendukung untuk membuat sistem aplikasi Internet of Things (IoT). Terlihat pada gambar di atas merupakan pin out dari ESP32. Pin tersebut dapat dijadikan input atau output untuk menyalakan LCD, lampu, bahkan untuk menggerakkan motor DC.

Pada pin out tersebut terdiri dari :

- 18 ADC (Analog Digital Converter, berfungsi untuk merubah sinyal analog ke digital)
- 2 DAC (Digital Analog Converter, kebalikan dari ADC)
- 16 PWM (Pulse Width Modulation)
- 10 Sensor sentuh
- 2 jalur antarmuka UART
- Pin antarmuka i2C, i2S, dan SPI

2. Pin pada Mikrokontroler

○ Pin Input

Pin input merupakan pin yang biasanya digunakan untuk menerima sinyal atau informasi dari perangkat eksternal seperti sensor, tombol, atau perangkat lainnya. Di dalam mikrokontroler, pin input ini dapat diatur sebagai input digital atau input analog, tergantung pada jenis sinyal yang akan diterima.

Pada umumnya perangkat mikrokontroler seperti ESP32, pin input dapat diberi nomor yang berbeda-beda tergantung pada model dan konfigurasi. Contoh pin input pada perangkat piranti cerdas seperti ESP32 bisa berupa pin yang ditunjukkan dengan label D0, D1, D2, dan seterusnya. Di samping itu, beberapa perangkat juga memiliki pin input analog yang ditunjukkan dengan label A0, A1, A2, dan seterusnya.

○ Pin Output

Pin output merupakan pin yang pasti ada pada tiap-tiap proto-board. Pin output digunakan untuk menghasilkan sinyal atau keluaran dari perangkat tersebut. Pin output biasanya digunakan untuk mengendalikan perangkat lain, seperti LED, motor, buzzer, relay, dan komponen lain yang memerlukan sinyal kontrol. Pin ini akan mengeluarkan tegangan sesuai dengan yang tertulis, biasanya 3V3 atau 5V. Pada mikrokontroler umumnya juga terdapat pin analog dan pin digital yang dapat kita

gunakan sebagai pin input atau pin output dengan menggunakan kode program Arduino IDE.

3. Arduino IDE

Arduino IDE adalah suatu IDE (Integrated Development Environment) yang digunakan untuk memprogram mikrokontroler berbasis Arduino. Di dalam software ini, pengguna menulis kode yang akan dijalankan oleh Arduino untuk melakukan berbagai fungsi. Pemrograman di Arduino IDE menggunakan bahasa pemrograman yang merupakan variasi dari C/C++ yang telah disederhanakan untuk memudahkan pengembangan. Bahasa pemrograman ini sering disebut sebagai **Arduino Language** atau **Arduino C++**.

Untuk pengembangan aplikasi, baik itu untuk Arduino atau mikrokontroler serupa seperti ESP32, pengguna menulis kode dalam bentuk **sketches**. Sketch adalah istilah yang digunakan di Arduino IDE untuk program yang ditulis oleh pengguna, dan file sketch disimpan dengan ekstensi **.ino**.

Fitur-fitur Arduino IDE:

1. **Editor Kode:** Tempat menulis dan mengedit sketch.
2. **Verifikasi/Compile:** Mengecek kesalahan sintaks dalam kode dan mengubah kode menjadi bahasa mesin.
3. **Upload:** Mengunggah kode ke mikrokontroler Arduino.
4. **Serial Monitor:** Mengirim dan menerima data dari Arduino melalui koneksi serial.
5. **Board Manager:** Menambahkan dukungan untuk berbagai jenis board.
6. **Library Manager:** Mengelola pustaka tambahan yang mungkin dibutuhkan untuk berbagai fungsi hardware atau protokol.

4. Fungsi-Fungsi Dasar Pemrograman Mikrokontroler

Kode program yang akan sering kita gunakan dapat dilihat pada tabel di bawah ini:

Fungsi	Deskripsi
setup()	Berfungsi untuk melakukan setup / inisialisasi program pada mikrokontroler. Fungsi ini akan berjalan 1x saja pada

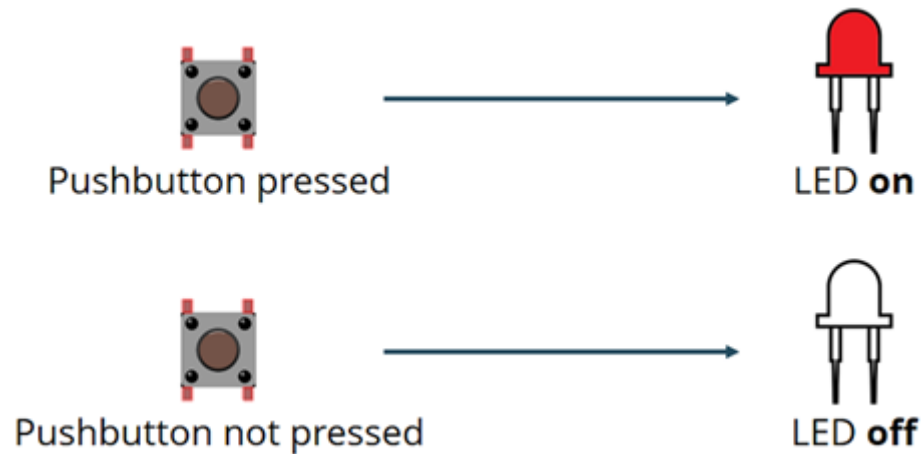
	saat pertama kali mikrokontroler dihidupkan.
loop()	Fungsi ini akan menjalankan semua kodenya secara terus menerus selama mikrokontroler hidup. Fungsi ini akan berjalan setelah semua kode pada setup selesai dikerjakan.
pinMode(pin, mode)	Berfungsi untuk merubah fungsi pin digital / analog pada mikrokontroler menjadi pin INPUT / pin OUTPUT. Saat memilih pin INPUT maka mikrokontroler hanya akan menerima tegangan dan daya pada pin tersebut dan tidak dapat mengeluarkan tegangan dan daya, hal ini akan berlaku sebaliknya saat memilih mode OUTPUT.
analogWrite(pin, value) digitalWrite(pin, value)	Tergantung tipe pin yang digunakan, kita harus menggunakan digital / analog write. Jika pin yang kita set sebagai output adalah pin analog, maka kita harus menggunakan analogWrite(), atau sebaliknya. Perbedaan kedua fungsi ini analogWrite() akan mengeluarkan tegangan dengan sinyal sinusoidal (rentang 0 – 1023) , dan digital akan mengeluarkan sinyal square (rentang 0 – 1).
delay()	Delay digunakan untuk melakukan jeda sebelum menjalankan kode pada baris berikutnya.
Serial.print()	Mencetak data ke port serial sebagai teks ASCII yang dapat dibaca. Perintah ini dapat mengambil banyak bentuk. Angka dicetak menggunakan karakter ASCII untuk setiap digit. Float dicetak dengan cara yang sama sebagai digit ASCII, defaultnya adalah dua tempat desimal. Bytes dikirim sebagai satu karakter. Karakter dan string dikirim apa adanya.
millis()	Mengembalikan jumlah milidetik yang berlalu sejak papan

	Arduino mulai menjalankan program saat ini.
micros()	Mengembalikan jumlah mikrodetik sejak papan Arduino mulai menjalankan program saat ini. Jumlah ini akan meluap (kembali ke nol), setelah kira-kira 70 menit.

5. Komponen: Switch



Push button switch adalah perangkat sederhana yang berfungsi untuk menghubungkan atau memutuskan aliran arus listrik dengan sistem kerja tekan unlock (tidak mengunci). Sistem kerja unlock disini berarti saklar akan bekerja sebagai device penghubung atau pemutus aliran arus listrik saat tombol ditekan, dan saat tombol tidak ditekan (dilepas), maka saklar akan kembali pada kondisi normal. Sebagai device penghubung atau pemutus, push button switch hanya memiliki 2 kondisi, yaitu On dan Off (1 dan 0).

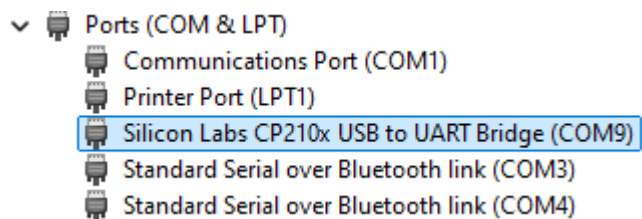


- **Studi Kasus: Persiapan Perangkat Lunak dan Perangkat Keras**

- 1. Memastikan Mikrokontroler Dapat Terhubung ke PC.**

- a. Hubungkan ESP32 ke PC.**

- Hubungkan ESP32 ke PC menggunakan kabel USB yang mendukung data (bukan hanya kabel pengisian daya).
- Pastikan LED di ESP32 menyala, yang berarti perangkat telah menerima daya.
- Pada sistem operasi Windows, lihat pada *Device Manager* jika Windows melakukan *auto-install* maka ESP32 bisa langsung dapat digunakan.



- b. Instal Driver.**

- Jika tidak muncul, instal terlebih dahulu *driver*-nya. Anda dapat mengunduh *driver* pada link berikut: [CP210x USB to UART Bridge VCP Drivers](#). (Umumnya *chip* usb-nya adalah CP2102, jika bukan, Anda bisa

mencari *driver* untuk yang lain, misal CH304).

Software Downloads

Software (11)

Software • 11

CP210x Universal Windows Driver

v11.3.0

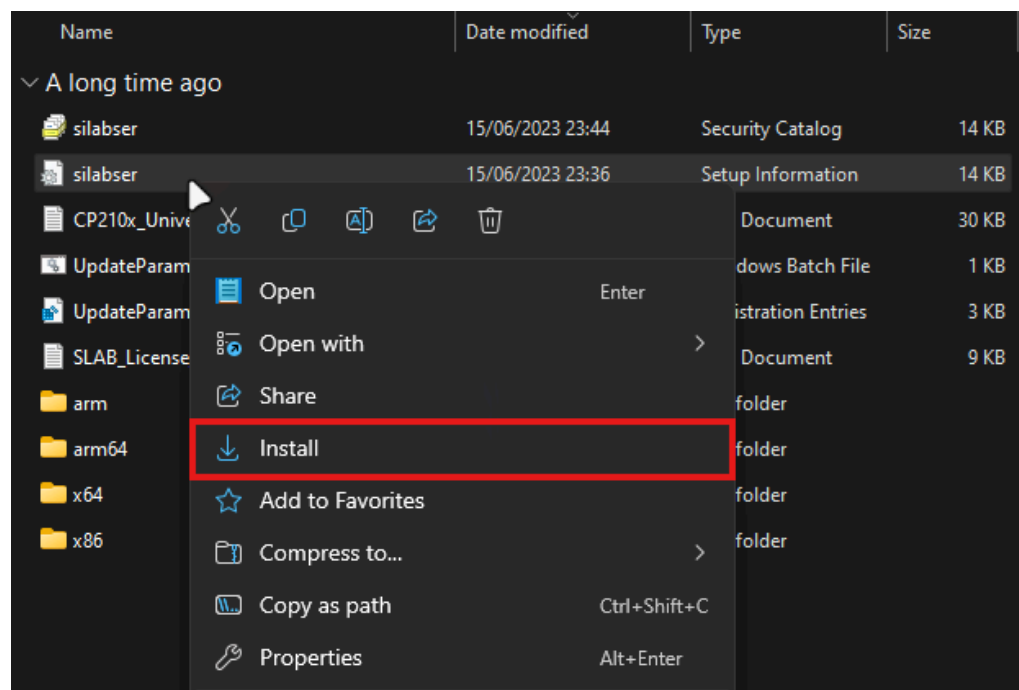
8/9/2024

CP210x VCP Mac OSX Driver

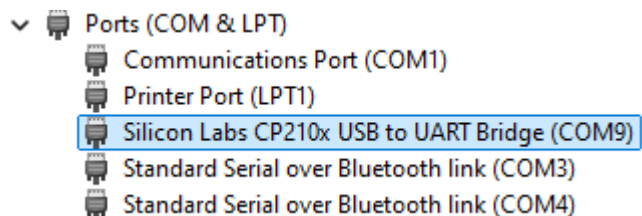
v6.0.2

10/27/2021

- Ekstrak **CP210x_Universal_Windows_Driver.zip** dan buka, kemudian pada file ***silabser.inf*** klik kanan dan klik install.



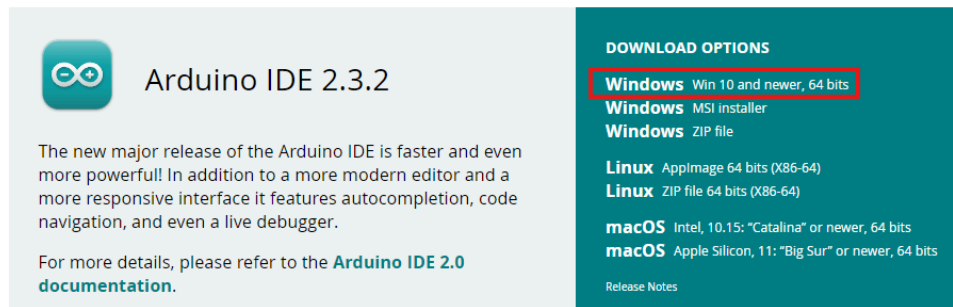
- Jika berhasil, maka CP210x akan muncul pada *Device Manager*, jika tidak bisa, coba cabut dan hubungkan kembali ke PC, jika masih tidak bisa kemungkinan *driver*-nya tidak cocok untuk mikrokontroler Anda.



2. Menginstal Arduino IDE dan Board Manager.

a. Unduh dan Instal Arduino IDE

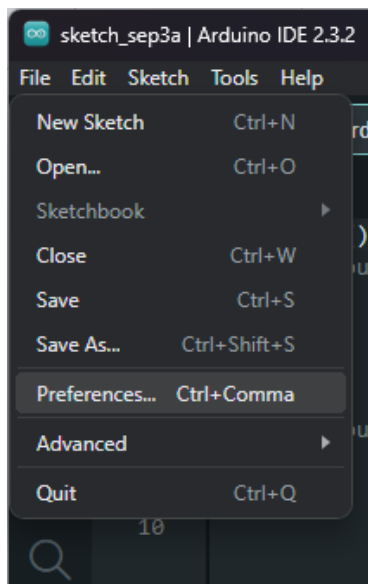
- Anda dapat mengunduh Arduino IDE pada website resminya <https://www.arduino.cc/en/software>.



- Kemudian instal Arduino IDE seperti biasa menginstal *software*.
- Buka Arduino IDE.

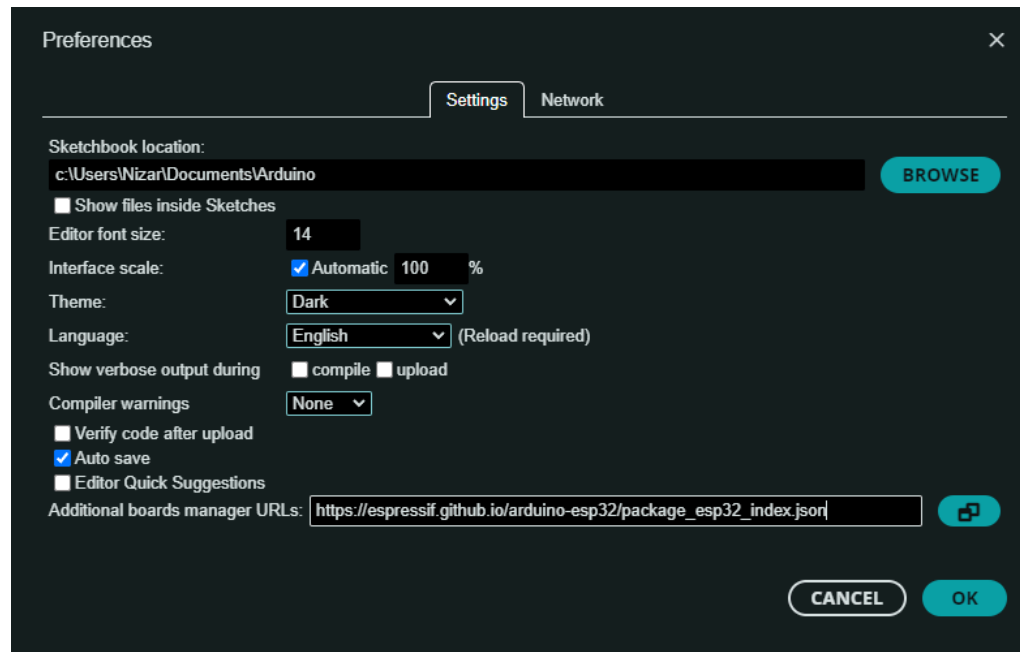
b. Menambahkan ESP32 Board Manager

- Pada Arduino IDE, klik **File > Preferences**.

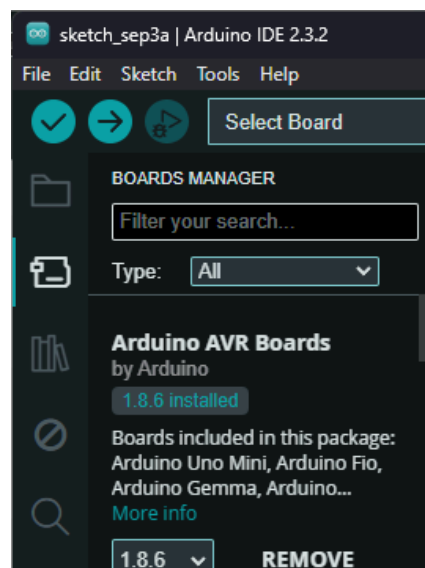


- Pada **Additional boards managers URLs** tambahkan link berikut:

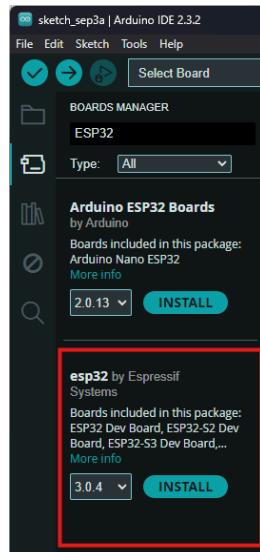
https://espressif.github.io/arduino-esp32/package_esp32_index.json



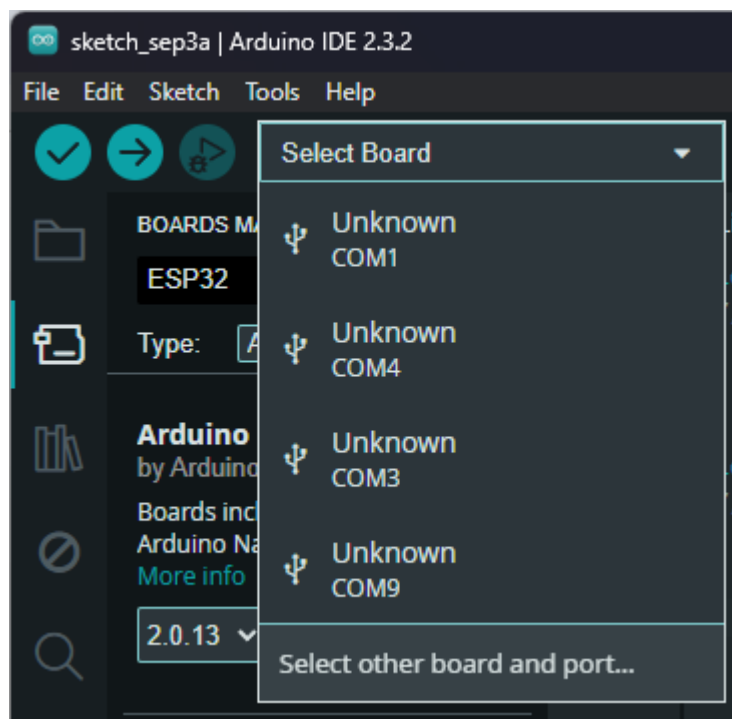
- Kemudian klik **OK**.
- Buka **board manager** yang terletak pada *sidebar* Arduino IDE.



- Cari ESP32 board manager yang baru ditambahkan tadi, yaitu yang di-publish oleh Espressif dan klik **INSTALL** tunggu sampai selesai.

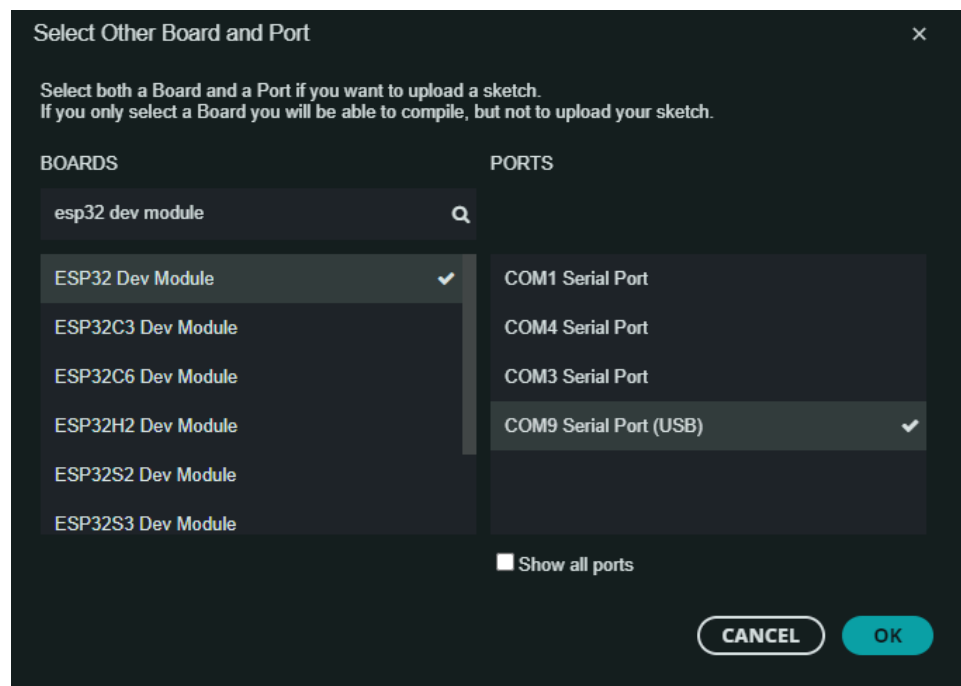


- Jika sudah, pastikan ESP32 sudah terhubung melalui USB kemudian klik **Select Board** dan klik **Select other board and port...**



- Pada bagian board, pilih ESP32 sesuai seri yang tertera pada mikrokontroler Anda, jika tidak yakin pilih saja ESP32 Dev Modul, dan pada bagian ports

sesuaikan dengan port pada **Device Manager**.



- Kemudian klik OK.
- Arduino IDE dan ESP32 siap digunakan.

BAB 2: PENGENALAN DIGITAL DAN ANALOG INPUT DAN OUTPUT

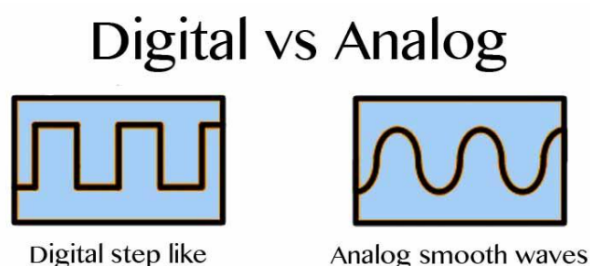
1. Pengenalan Digital dan Analog Input dan Output

Dalam teknologi modern, pemahaman tentang input dan output digital serta analog memiliki peran penting dalam pengembangan sistem elektronik dan komputer. Input digital merujuk pada sinyal diskrit seperti data dari tombol keyboard atau saklar elektronik, yang diolah oleh perangkat digital. Output digital, yang hanya memiliki nilai 0 atau 1, mengendalikan perangkat seperti lampu atau layar.

Input analog melibatkan sinyal kontinu seperti suhu atau tekanan, memerlukan konversi menjadi format digital untuk diolah dalam sistem digital. Output analog mencerminkan variasi kontinu, contohnya gelombang suara dari alat audio. Pemahaman akan output analog krusial dalam pemrosesan suara dan pengendalian motor.

Gabungan input dan output digital serta analog menjadi dasar sistem kontrol, sensor, dan komunikasi. Memahami perbedaan dan hubungan keduanya memungkinkan para profesional merancang sistem yang terintegrasi secara efisien, mendorong inovasi di berbagai bidang kehidupan modern.

2. I/O Digital dan Analog



Gambar di atas menyajikan representasi visual dari gelombang digital dan analog. Sensor dapat didesain dengan baik digital dan analog ataupun salah satu dari mereka saja. Bahkan, tersedia akselerometer yang analog dan digital dimana sensor cahaya dan suara dianggap sebagai analog. Dalam piranti cerdas memiliki berbagai macam perangkat output yang dapat diimplementasikan dalam kehidupan sehari-hari. Seperti yang sudah dijelaskan di atas bahwa sebuah data input akan menghasilkan suatu tindakan berupa output. Misal sensor cahaya (input) dengan output fisik yaitu

cahaya pada sebuah LED dll. Namun pada praktikum kali ini , kita akan menggunakan kedua i/o yaitu i/o digital dan analog, antara lain:

Perangkat input:

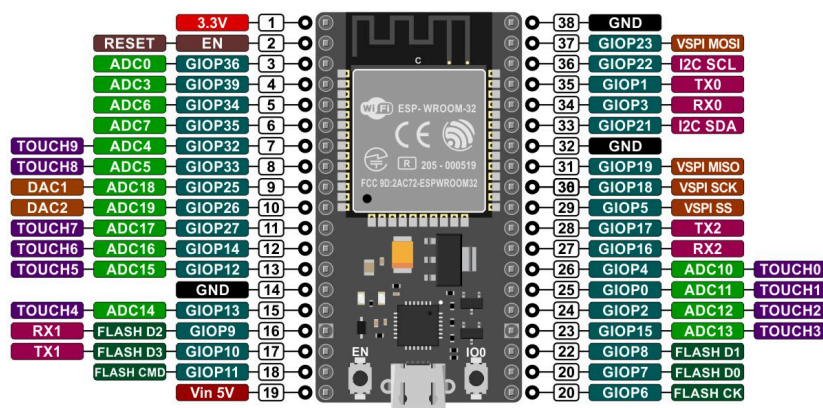
- a. MPU6050
- b. Light Dependent Resistor (LDR)

Perangkat output:

- a. LED
- b. Buzzer

a. ADC (analog to digital Converter) dan DAC (Digital To Analog Converter)

Fitur ADC (analog to digital Converter) dan DAC (Digital To Analog Converter) spesifik dapat digunakan hanya pada pin -pin tertentu saja. Sedangkan fitur UART, I2C, SPI, PWM dapat diaktifkan secara programmatically. Berikut adalah diagram pin-pin pada development Board ESP32



Meskipun begitu, tidak semua pin dengan fitur tertentu pada NodeMCU ESP32 cocok digunakan untuk semua keperluan di dalam project. Masing-masing pin memiliki fungsinya tersendiri, misalnya pin-pin mana yang baik digunakan sebagai input atau pin-pin mana yang baik digunakan sebagai output. Pin yang diberi highlight hijau, bisa digunakan di dalam project. Sedangkan pin dengan highlight kuning bisa digunakan namun dengan catatan yang perlu diperhatikan, karena terdapat perilaku yang tak terduga terutama saat proses boot. Pin dengan highlight merah tidak direkomendasikan sebagai input maupun output. Untuk lebih memahami

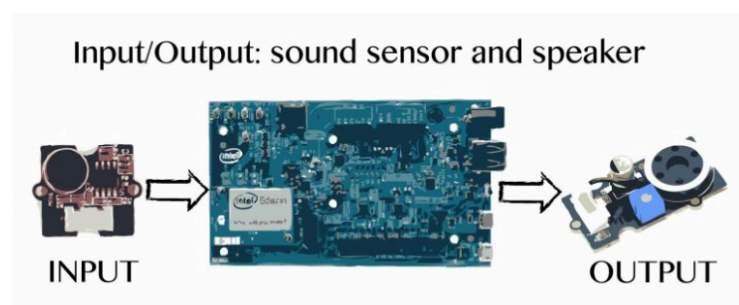
fungsi-fungsi pin pada gambar di atas , mari simak table di bawah ini :

GPIO	Input	Output	Catatan
0	Pulled up	OK	output sinyal PWM saat boot
1	TX pin	OK	output debug saat boot
2	OK	OK	Terhubung ke LED on board
3	OK	TX pin	HIGH saat boot
4	OK	OK	
5	OK	OK	output sinyal PWM saat boot
6	x	x	terhubung dengan SPI Flash terintegrasi
7	x	x	terhubung dengan SPI Flash terintegrasi
8	x	x	terhubung dengan SPI Flash terintegrasi
9	x	x	terhubung dengan SPI Flash terintegrasi
10	x	x	terhubung dengan SPI Flash terintegrasi
11	x	x	terhubung dengan SPI Flash terintegrasi
12	OK	OK	boot gagal ketika mendapatkan input high
13	OK	OK	
14	OK	OK	output sinyal PWM saat boot
15	OK	OK	output sinyal PWM saat boot
16	OK	OK	
17	OK	OK	
18	OK	OK	
19	OK	OK	
20	OK	OK	
21	OK	OK	
22	OK	OK	
23	OK	OK	
24	OK	OK	
25	OK	OK	
26	OK	OK	
27	OK	OK	

28	OK	OK	
29	OK	OK	
30	OK	OK	
31	OK	OK	
32	OK	OK	
33	OK	OK	
34	OK	OK	
35	OK	OK	
36	OK		Hanya input
37	OK		Hanya input
38	OK		Hanya input
39	OK		Hanya input

3. Perangkat Input Output

Input dan output (I/O) adalah kunci untuk mendalami pembuatan prototipe perangkat piranti cerdas. Perangkat input berperan untuk merasakan berbagai perbedaan yang terjadi di dunia nyata seperti temperatur, sentuhan, gaya, kelembaban, dan medan magnet. Agar sebuah sirkuit dapat melakukan sebuah tindakan yang berguna, sirkuit tersebut harus mendapatkan data (*input*) yang mana ia akan meresponnya dengan sebuah tindakan (*output*). Fungsi *output* biasanya diurus oleh aktuator, yang mana mengontrol perangkat seperti lampu atau speaker.

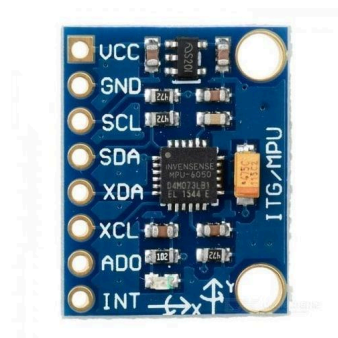


Gambar diatas menunjukkan skenario I/O di mana sebuah sensor suara (*input*) mengkonversi suara menjadi impuls listrik dan sebuah speaker (*output*) mengkonversikan impuls tersebut menjadi suara. Alurnya mulai dari sebuah *input*, diikuti oleh proses mengubah energi fisik menjadi gelombang elektrik, yang mengarah ke *output* fisik (suara). Ketika sebuah sensor mendeteksi satu atau lebih

sinyal (input), ia akan mengkonversikan sinyal tersebut menjadi representasi analog atau digital untuk diterima oleh aktuator. Data digital adalah hitungan presisi yang membuat grafik berirama dengan penaikan yang tajam, bagian konstan, dan penurunan yang tajam. Contoh pada gambar diatas datanya adalah analog. Berikut adalah perangkat input dan output digital serta analog:

a. Perangkat Input

1. MPU6050



Sensor MPU6050 adalah sensor mampu membaca kemiringan sudut berdasarkan data dari sensor accelerometer dan sensor gyroscope. Modul MPU6050 adalah papan elektronik yang mengintegrasikan kedua elemen tersebut untuk memungkinkan kita dapat mengukur perubahan posisi elemen sehingga menghasilkan suatu reaksi misalnya seperti, ketika suatu objek bergerak, LED menyala, atau hal-hal lain yang jauh lebih rumit. Sensor ini juga dilengkapi oleh sensor suhu yang dapat digunakan untuk mengukur suhu di keadaan sekitar. Modul MPU6050 ini memiliki 6 sumbu kebebasan, DoF, Akselerometer akselerasi X, Y, dan Z 3 sumbu, dan giroskop 3 sumbu lainnya untuk mengukur kecepatan sudut. Jalur data yang digunakan pada sensor ini adalah jalur data I2C. Untuk bekerja dengan kedua arah di bus I2C, kita dapat menggunakan pin dan petunjuk arah ini :

AD0 = 1 atau High (5v): untuk alamat I0C 69x2.

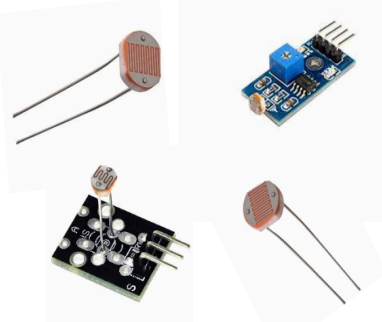
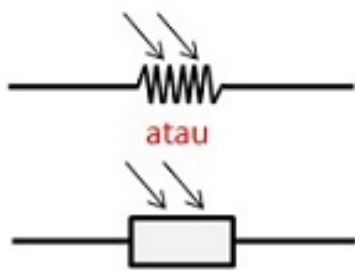
AD0 = 0 atau Low (GND atau Nc): untuk alamat 0x68 dari bus I2C.

Catatan : Resistansi internal ke GND, jika pin ini tidak terhubung maka alamatnya secara default akan menjadi 0x68, karena ini akan terhubung

secara default dan ditafsirkan sebagai logika 0.

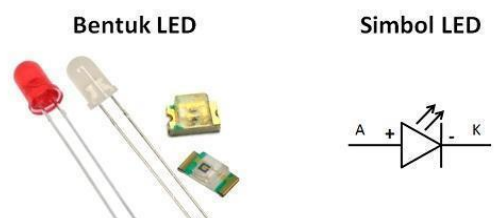
2. LIGHT DEPENDENT RESISTOR (LDR)

LDR (Light Dependent Resistor) merupakan salah satu komponen resistor yang nilai resistansinya akan berubah-ubah sesuai dengan intensitas cahaya yang mengenai sensor ini. Jadi bisa dikatakan bahwa LDR (Light Dependent Resistor) atau photoresistor ini merupakan komponen yang sensitif terhadap cahaya. Semakin banyak cahaya yang mengenainya, maka akan semakin menurun nilai resistansinya. Sebaliknya jika semakin sedikit cahaya yang mengenai sensor (gelap), maka nilai hambatannya akan menjadi semakin besar sehingga arus listrik yang mengalir akan terhambat. Umumnya Sensor LDR memiliki nilai hambatan 200 Kilo Ohm pada saat dalam kondisi sedikit cahaya (gelap), dan akan menurun menjadi 500 Ohm pada kondisi terkena banyak cahaya. Tak heran jika komponen elektronika peka cahaya ini banyak diimplementasikan sebagai sensor lampu penerang jalan, lampu kamar tidur, alarm dan lain-lain.

Bentuk LDR	Simbol LDR
	

b. Perangkat Output

1. LED



LED atau singkatan dari Light Emitting Diode adalah komponen elektronika yang dapat memancarkan cahaya apabila mendapatkan aliran tegangan maju,

atau dialiri tegangan layaknya dioda dengan konfigurasi tegangan maju. Dilihat dari simbol LED juga hampir sama dengan simbol dioda, hanya saja memiliki sedikit perbedaan pada simbol yang menjelaskan bahwa komponen ini dapat memancarkan cahaya apabila mendapatkan aliran tegangan. Bahan semikonduktor yang digunakan untuk membuat led dapat memberikan bermacam warna pada led, misalnya warna merah, hijau, kuning, dan warna biru. Ada pula jenis led yang dapat difungsikan untuk memancarkan cahaya infrared misalnya dalam penggunaan untuk remote control.

2. BUZZER

Bentuk BUZZER	Simbol BUZZER
	

Buzzer adalah komponen elektronika yang dapat menghasilkan getaran suara dalam bentuk gelombang bunyi. Buzzer lebih sering digunakan karena ukuran penggunaan dayanya yang minim. Prinsip kerja buzzer adalah sangat sederhana, Ketika suatu aliran listrik mengalir ke rangkaian buzzer, maka terjadi pergerakan mekanis pada buzzer tersebut. Akibatnya terjadi perubahan energi dari energi listrik menjadi energi suara yang dapat didengar oleh manusia. Umumnya jenis buzzer yang beredar di pasaran adalah buzzer piezoelectric yang bekerja pada tegangan 3 sampai 12 volt DC.

Jenis-jenis buzzer pada rangkaian Arduino berdasarkan bunyinya terbagi atas dua, yaitu:

- Active Buzzer, yaitu buzzer yang sudah memiliki suaranya sendiri saat diberikan tegangan listrik. Buzzer aktif Arduino jenis ini seringkali juga disebut buzzer stand alone atau berdiri sendiri.
- Passive Buzzer, yaitu buzzer yang tak memiliki suara sendiri. Buzzer jenis ini sangat cocok dipadukan dengan Arduino karena kita bisa memprogram tinggi rendah nadanya. Salah satu contohnya adalah speaker.

Referensi: <https://wiraelectrical.com/id/digital-i-o-analog-i-o/>

BAB 3: KONEKSI WIFI

1. WiFi pada ESP32

Untuk menghubungkan perangkat keras ESP32 dengan jaringan Wi-Fi, dapat digunakan *library* WiFi (**WiFi.h**) dari Arduino. Pada *library* WiFi ada empat metode yang akan sering digunakan:

- **WiFi.status()**: melihat koneksi Wi-Fi.
- **WiFi.localIP()**: melihat alamat IP dari router yang terhubung.
- **WiFi.RSSI()**: memperoleh nilai kuat sinyal Wi-Fi.
- **WiFi.begin()**: memulai hubungan dengan jaringan Wi-Fi.

a. Menghubungkan ke Jaringan WiFi

Langkah penting dalam mengkoneksikan ESP32 dengan jaringan WiFi adalah melalui fungsi **WiFi.begin()**. Proses ini memungkinkan ESP32 untuk menginisialisasi koneksi ke jaringan WiFi yang diinginkan. Konsep dasar dari fungsi ini melibatkan penggunaan **SSID (Service Set Identifier)** atau nama jaringan WiFi dan juga **password WiFi** sebagai informasi autentikasi.

Konsep dasar fungsi **WiFi.begin()**:

- **SSID (Service Set Identifier)**: Ini adalah nama unik yang mengidentifikasi jaringan WiFi tertentu. SSID diperlukan agar perangkat tahu ke jaringan mana harus terhubung.
- **Password WiFi**: Ini adalah kata sandi yang diperlukan untuk mengautentikasi perangkat ke jaringan WiFi yang aman. Hanya perangkat dengan password yang tepat yang diizinkan untuk terhubung.

Contoh program:

```
1 #include <WiFi.h>
2
3 const String ssid = "hotspot";
4 const String password = "12345678";
5
6 void setup() {
7   Serial.begin(9600);
8   WiFi.begin(ssid, password);
9
10  while(WiFi.status() != WL_CONNECTED){
11    delay(1000);
12    Serial.println("Menghubungkan ke WiFi...");
13  }
14
15  Serial.println("Terhubung ke WiFi " + ssid);
16 }
17
18 void loop() {
19
20 }
```

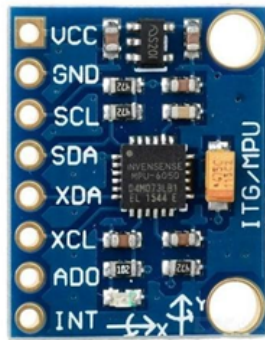
b. Memeriksa Status Koneksi WiFi pada ESP32

Proses memeriksa status koneksi WiFi pada modul ESP32 sangat penting dalam pengembangan piranti cerdas. Status ini memberikan informasi tentang apakah modul terhubung ke jaringan WiFi atau tidak. Fungsi yang umum digunakan untuk memeriksa status ini adalah **WiFi.status()**. Pada **WiFi.status()** ada beberapa macam status dari koneksi Wi-Fi:

Nilai	Konstan	Keterangan
0	WL_IDLE_STATUS	Mikrokontroler dalam keadaan bebas atau idle, tidak terhubung ke jaringan.
1	WL_NO_SSID_AVAIL	Tidak ada jaringan WiFi dengan SSID yang dicari.
2	WL_SCAN_COMPLETED	Pencarian jaringan WiFi di sekitar modul telah selesai.
3	WL_CONNECTED	Modul terhubung ke jaringan WiFi dengan sukses.
4	WL_CONNECT_FAILED	Gagal menghubungkan ke jaringan WiFi.
5	WL_CONNECTION_LOST	Koneksi WiFi terputus.

6	WL_DISCONNECTED	Mikrokontroler terputus dari jaringan WiFi.
255	WL_NO_SHIELD	Tidak ada jaringan Wi-Fi yang tersedia di sekitar perangkat.

2. Komponen: MPU6050



Sensor MPU6050 adalah sensor mampu membaca kemiringan sudut berdasarkan data dari sensor accelerometer dan sensor gyroscope. Modul MPU6050 adalah papan elektronik yang mengintegrasikan kedua elemen tersebut untuk memungkinkan kita dapat mengukur perubahan posisi elemen sehingga menghasilkan suatu reaksi misalnya seperti, ketika suatu objek bergerak, LED menyala, atau hal-hal lain yang jauh lebih rumit.

3. Komponen: Buzzer



Buzzer adalah komponen elektronika yang dapat menghasilkan getaran suara dalam bentuk gelombang bunyi. Buzzer lebih sering digunakan karena ukuran penggunaan dayanya yang minim. Prinsip kerja buzzer adalah sangat sederhana, Ketika suatu aliran listrik mengalir ke rangkaian buzzer, maka terjadi pergerakan mekanis pada buzzer tersebut. Akibatnya terjadi perubahan energi dari energi listrik menjadi energi suara yang dapat didengar oleh manusia.

BAB 4: PROTOKOL KOMUNIKASI

1. HTTP

HTTP (Hypertext Transfer Protocol) adalah protokol komunikasi yang penting dalam lingkungan web. Dalam pengembangan web, metode HTTP yang umum digunakan adalah POST dan GET. Metode ini memungkinkan interaksi antara peramban web dan server, memfasilitasi pertukaran data dan informasi.

HTTP GET, sebagai salah satu metode dalam HTTP, digunakan untuk mengambil data dari server. Ketika permintaan GET dikirimkan oleh peramban web, server akan merespons dengan data yang diminta. Data ini dapat berupa halaman web, gambar, video, atau berbagai jenis konten lainnya. Permintaan GET dapat mengandung parameter dalam URL untuk mengarahkan permintaan dengan lebih spesifik, sehingga memungkinkan perolehan data yang diinginkan. Namun, karena parameter URL terbuka, informasi sensitif dapat terekspos jika tidak dikelola dengan benar.

HTTP POST, di sisi lain, digunakan untuk mengirim data dari peramban web ke server. Data ini biasanya tidak ditampilkan langsung dalam URL seperti pada permintaan GET, sehingga lebih cocok digunakan untuk mengirim informasi sensitif seperti kata sandi atau data pribadi. Permintaan POST mengirimkan data dalam badan permintaan HTTP, yang tidak terlihat oleh pengguna biasa. Hal ini membuatnya lebih aman untuk pengiriman data sensitif. Server akan merespons permintaan POST dengan memberikan balasan sesuai, yang dapat berupa konfirmasi sukses atau pesan kesalahan jika ada.

Secara keseluruhan, HTTP GET dan POST adalah dua metode utama dalam protokol HTTP yang memungkinkan komunikasi dan pertukaran data antara peramban web dan server. Pemahaman yang baik tentang perbedaan antara keduanya penting dalam pengembangan web yang efektif dan aman.

2. Pengiriman Data Ke Database Dengan Protokol HTTP

a. Database

Database atau database adalah koleksi terstruktur informasi yang disimpan secara elektronik dalam sistem komputer. Ini memiliki peran penting dalam pengelolaan data

untuk berbagai tujuan, dari bisnis hingga penelitian. Komponen utamanya mencakup entitas, atribut, dan relasi di antara entitas tersebut. Database Management System (DBMS) adalah perangkat lunak yang memfasilitasi pengelolaan, akses, dan manipulasi data dalam database. Jenis database seperti relasional, NoSQL, dan berorientasi grafik telah dikembangkan untuk kebutuhan yang bervariasi dalam skala dan kompleksitas.

Dalam piranti cerdas, database penting untuk mengelola data dari perangkat terhubung seperti sensor dan perangkat pintar. Data yang dihasilkan sangat besar dan realtime. Database harus bisa menangani aliran data terus-menerus dan mendukung penyimpanan serta pencarian data efisien. database juga harus bisa memproses data secara paralel dan dapat ditingkatkan seiring bertambahnya perangkat dan data. Penggunaan database yang tepat memungkinkan analisis pola, prediksi, dan pengambilan keputusan yang lebih baik. Namun, perlindungan data dan privasi juga penting dalam pengembangan database untuk piranti cerdas guna menjaga keamanan data dalam lingkungan yang kompleks dan terhubung.

b. Pengiriman Data ke Database

Pengiriman data ke database melalui protokol HTTP merupakan hal yang umum dalam piranti cerdas. Protokol HTTP digunakan dalam World Wide Web (www) untuk komunikasi data antara server dan klien. Ini memfasilitasi piranti cerdas mengirimkan informasi ke database secara efisien melalui internet, termasuk data sensor, status perangkat, dan interaksi pengguna. Penggunaan HTTP mempermudah akses data dari berbagai platform serta integrasi antara piranti cerdas dan sistem lainnya.

Meski pengiriman data ke database melalui HTTP memiliki keuntungan, seperti implementasi mudah dan interoperabilitas yang baik, kita perlu mempertimbangkan aspek tertentu. Keamanan data saat pengiriman menjadi salah satu pertimbangan penting. Karena data yang dikirim bisa berisi informasi sensitif, perlu ada mekanisme keamanan seperti enkripsi dan otentikasi untuk melindungi data dari akses yang tidak sah. Selain itu, pengiriman data melalui HTTP membutuhkan jaringan yang stabil agar data dapat dikirim dan diterima dengan baik, terutama di lingkungan piranti cerdas yang terus terhubung.

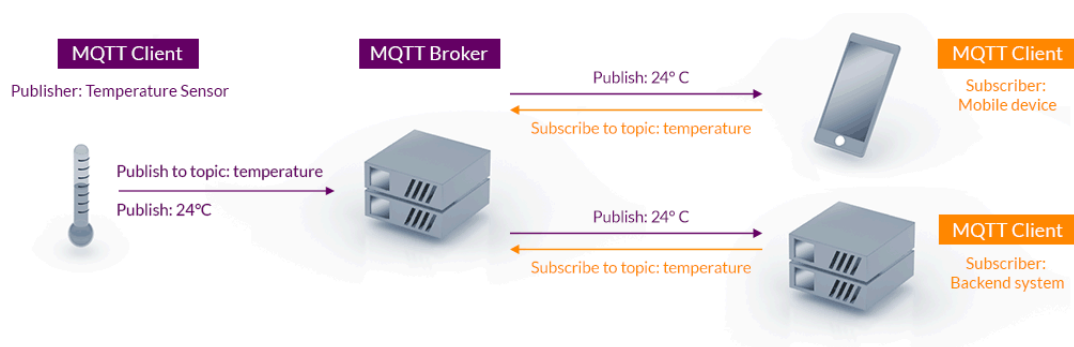
3. MQTT

MQTT adalah protokol jaringan ringan yang menggunakan model *publish-subscribe* untuk komunikasi antar mesin (*machine-to-machine*). Protokol ini dirancang khusus untuk koneksi dengan perangkat di lokasi terpencil yang memiliki keterbatasan sumber daya atau *bandwidth* jaringan terbatas, seperti yang umum ditemui dalam ekosistem *Internet of Things* (IoT).

Dengan kata lain, MQTT memungkinkan pertukaran pesan secara efisien dan handal, bahkan pada perangkat dengan daya komputasi dan koneksi internet yang minim seperti ESP32. Model *message queuing* yang diusungnya memungkinkan pengiriman pesan secara *asynchronous*, artinya pengirim dan penerima pesan tidak perlu terhubung secara langsung atau bersamaan.

a. Arsitektur MQTT: Pub/Sub Pattern

MQTT menggunakan pola *publish-subscribe* (*pub/sub*) untuk komunikasi. Dalam model ini, pengirim pesan (*publisher*) dan penerima pesan (*subscriber*) tidak berinteraksi langsung. Sebuah perantara, yang disebut broker MQTT, bertugas menerima pesan dari *publisher*, memfilternya berdasarkan topik, dan meneruskannya ke *subscriber* yang *subscribe* (berlangganan) topik tersebut. Hal ini memungkinkan komunikasi yang efisien dan terdesentralisasi, terutama untuk perangkat IoT dengan sumber daya terbatas.



b. Client dan Broker (Server)

Client MQTT adalah perangkat apa pun, mulai dari mikrokontroler kecil hingga server yang besar, yang menjalankan *library* MQTT dan terhubung ke broker. Client dapat berperan sebagai *publisher*, *subscriber*, atau keduanya.

Sedangkan **Broker MQTT** adalah pusat dari komunikasi MQTT. Broker menerima semua pesan dari *client publisher*, memfilternya berdasarkan topik, dan mengirimkannya ke *client subscriber* yang tepat. Broker juga bertanggung jawab untuk otentikasi dan otorisasi *client*, serta menyimpan sesi dan pesan untuk client yang terputus sementara (jika dikonfigurasi demikian). Salah satu Broker MQTT yang terkenal adalah **Eclipse Mosquitto**.

Sama seperti HTTP, koneksi MQTT didasarkan pada TCP/IP. Client memulai koneksi dengan mengirim pesan CONNECT ke broker. Broker merespons dengan pesan CONNACK untuk mengkonfirmasi koneksi. Koneksi tetap terbuka sampai client mengirim pesan DISCONNECT atau koneksi terputus.

c. Topik

Topik dalam MQTT adalah string UTF-8 yang digunakan untuk memfilter pesan. Topik memiliki struktur hierarkis, dipisahkan oleh garis miring (/). Contoh: rumah/lantai_atas/kamar_tidur/suhu.

Client *publish* pesan ke topik tertentu. Client *subscribe* ke topik yang ingin mereka terima pesannya. Broker akan mengirimkan pesan yang di-*publish* ke topik tertentu hanya kepada client yang *subscribe* ke topik tersebut atau topik yang cocok dengan wildcard. Terdapat dua wildcard:

- **Single-level wildcard (+):** Menggantikan satu level topik. Contoh: berlangganan ke sensor/+/suhu akan menerima pesan dari sensor/1/suhu, sensor/2/suhu, dst.
- **Multi-level wildcard (#):** Menggantikan beberapa level topik dan harus diletakan di akhir. Contoh: berlangganan ke sensor/# akan menerima pesan dari semua topik yang dimulai dengan sensor/.

Agar sistem MQTT dapat mudah dikelola, berikut beberapa praktik terbaik (*best practices*) dalam penulisan topik:

- **Hindari garis miring di awal:** garis miring di awal topik (/rumah/ruang_tamu/suhu) hanya menambah kompleksitas.
- **Hindari spasi:** Spasi dalam topik dapat menyulitkan debugging dan integrasi dengan sistem lain.

- **Topik yang singkat dan padat:** Topik yang panjang akan menambah ukuran pesan dan memperlambat komunikasi.
- **Gunakan karakter ASCII:** Meskipun MQTT support UTF-8, karakter non-ASCII dapat menyebabkan masalah tampilan dan kompatibilitas.

d. Quality of Service (QoS)

Dalam jaringan yang tidak selalu dapat diandalkan, penting untuk memiliki mekanisme yang menjamin pengiriman pesan. Di sinilah peran *Quality of Service* (QoS) dalam MQTT. QoS mendefinisikan tingkat jaminan pengiriman pesan antara pengirim dan penerima. MQTT menyediakan tiga tingkat QoS:

- **QoS 0 (At Most Once):** Pada tingkat ini, pesan dikirim dengan upaya terbaik, tanpa jaminan akan sampai. Tidak ada mekanisme *acknowledgement* atau pengiriman ulang. Cocok untuk data yang tidak kritis atau dikirim secara berkala yang kehilangan beberapa pesan tidak menjadi masalah.
- **QoS 1 (At Least Once):** QoS 1 menjamin pesan akan sampai setidaknya sekali, tetapi mungkin terkirim lebih dari sekali (duplikat). Pengirim akan menyimpan pesan dan mengirim ulang sampai menerima konfirmasi dari penerima. Cocok untuk kebanyakan kasus yang kehilangan pesan tidak dapat ditoleransi, tetapi duplikat pesan masih dapat diatasi.
- **QoS 2 (Exactly Once):** QoS 2 menjamin pesan akan sampai tepat sekali. Ini dicapai melalui mekanisme *handshake* empat arah yang lebih kompleks. QoS 2 memberikan jaminan pengiriman tertinggi, tetapi juga memiliki *overhead* yang lebih tinggi dan waktu pengiriman yang lebih lama. Cocok untuk data yang sangat penting yang duplikat pesan sama sekali tidak dapat ditoleransi.

Broker MQTT akan menggunakan tingkat QoS terendah di antara keduanya. Misalnya, jika *publisher* mengirim pesan dengan QoS 2, tetapi *subscriber* berlangganan dengan QoS 1, maka *broker* akan mengirimkan pesan tersebut dengan QoS 1. Pada studi kasus di bawah akan menggunakan *library* PubSubClient yang hanya dapat *publish* pesan QoS 0 dan dapat *subscribe* pada QoS 0 atau QoS 1. Anda dapat mencari *library* yang mendukung *pub-sub* QoS 0/1/2 lain jika perlu.

e. Studi Kasus: Mencoba MQTT

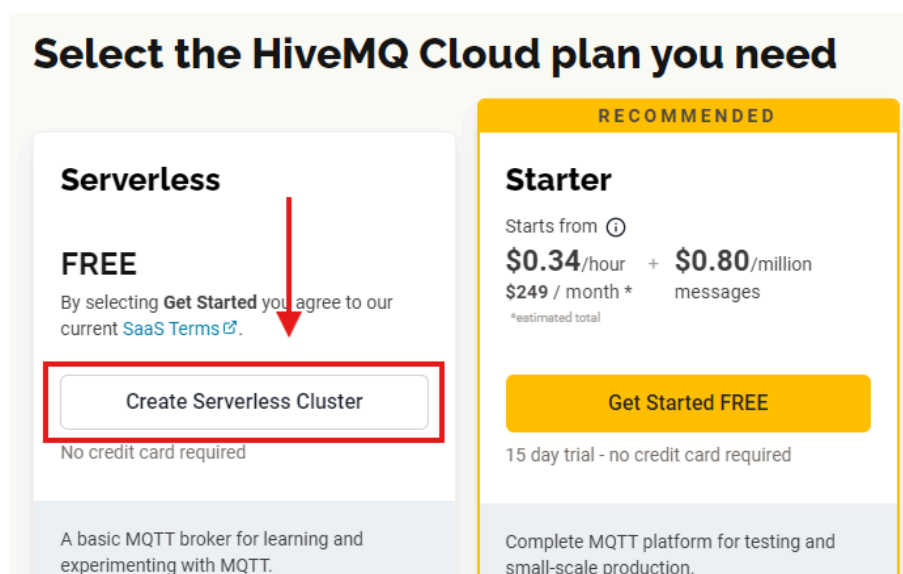
i. Membuat broker di HiveMQ

Pada studi kasus ini, kita akan membuat broker dengan platform HiveMQ, sehingga kita tidak perlu menginstal broker seperti Mosquitto pada PC/Laptop kita. Kunjungi <https://www.hivemq.com/> dan buatlah akun sesuai petunjuk di web tersebut.

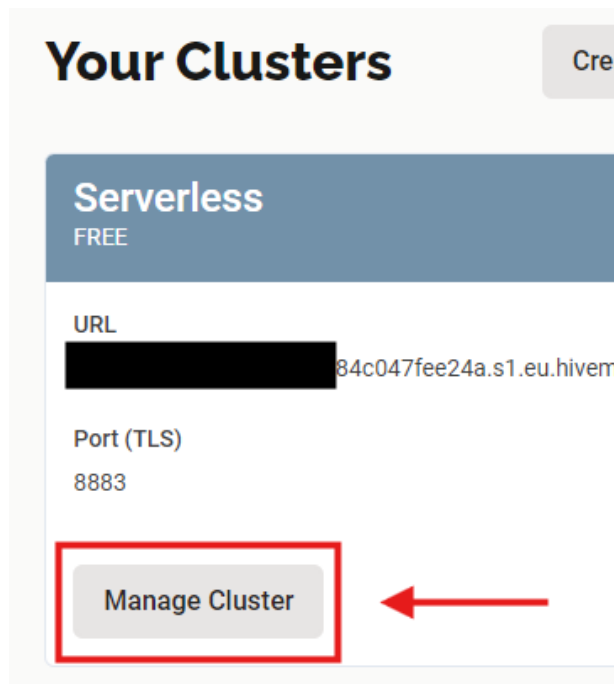
- Jika sudah sampai pada halaman ini, klik **Create New Cluster**.



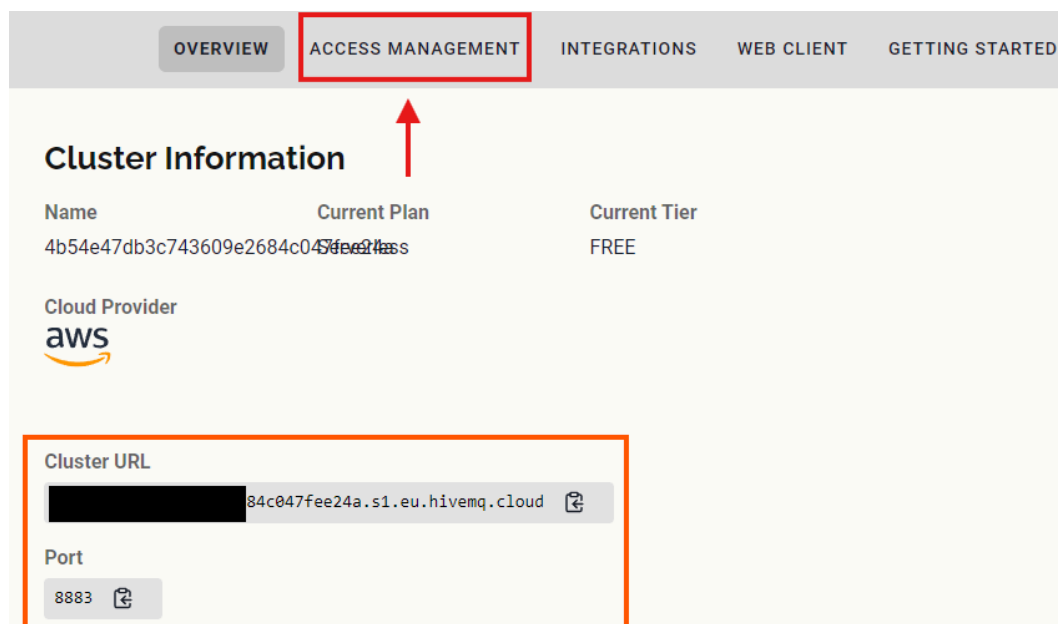
- Kemudian pada halaman selanjutnya, klik **Create Serverless Cluster**.



- Kemudian jika berhasil akan muncul cluster baru. Klik **Manage Cluster**.



- Selanjutnya catatlah Cluster URL dan Portnya, ini akan digunakan pada kode di mikrokontroler. Jika sudah, klik **Access Management**.



- Pada bagian Authentication, klik **Edit** pada Credentials dan **Add Credentials**, kemudian buatlah username beserta passwordnya, dengan permission **Publish and Subscribe**. Catat untuk digunakan pada kode nanti.

 **Authentication**

Credentials

Hide

Currently you have not created any credentials. Fill out the following form to create an access credentials pair and limit access to your HiveMQ Cloud MQTT instance. To learn more check out our Security Fundamentals guide.

Username *

esp32

Permission *

Publish and Subscribe

▼

Password *

rahasia123



Confirm Password *

rahasia123

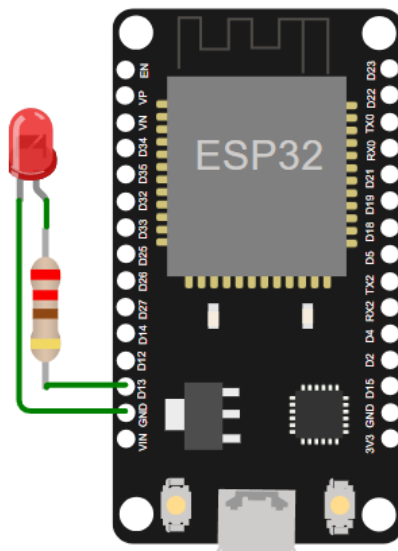


Save

Cancel

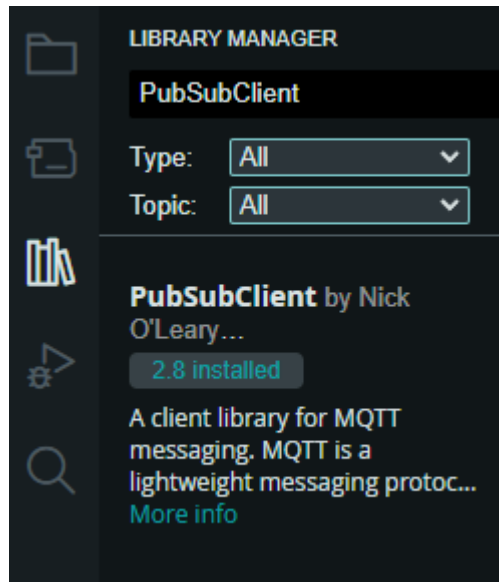
ii. Rangkaian

- Buatlah rangkaian seperti berikut, tentunya sesuaikan pada breadboard Anda.



iii. Kode

- Pada Arduino IDE tambahkan *library* PubSubClient, klik **Library Manager** kemudian ketikkan PubSubClient kemudian install *library* PubSubClient oleh Nick O'Leary.



- Kemudian tuliskan kode berikut pada Arduino IDE anda dan upload kode, jangan lupa pastikan mikrokontroler Anda sudah terkoneksi. Sesuaikan SSID dan password WiFi juga *cluster url* broker HiveMQ, dan *username-password* yang sudah Anda buat.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <WiFiClientSecure.h>

const int ledPin = 13;

//----- WiFi settings
const char* ssid = "isi-ssid";
const char* password = "isi-password";

//----- HiveMQ Cloud Broker settings
const char* mqtt_server = "Isi-Cluster-URL.hivemq.cloud";
const char* mqtt_username = "isi-username";
const char* mqtt_password = "isi-password";
const int mqtt_port = 8883;
const char* mqttSubscribeTopic = "esp32/led/control";
const char* mqttPublishTopic = "esp32/led/status";
```

```
WiFiClientSecure espClient;  
PubSubClient client(espClient);
```

```
// HiveMQ Cloud Let's Encrypt CA certificate
```

```
static const char *root_ca PROGMEM = R"EOF(  
-----BEGIN CERTIFICATE-----  
MIIFazCCA10gAwIBAgIRAIQz7DSQONZRGPgu20CiwAwDQYJKoZIhvcNAQELBQAw  
TzELMAkGA1UEBhMCVVMxKTAnBgNVBAoTIEludGVybmV0IFNlY3VyaXR5IFJlc2Vh  
cmNoIEdyb3VwMRUwEwYDVQQDEwxFb3VyaXR5IFNlY3VyaXR5IFJlc2VhcmNoIE  
dyb3VwMRUwEwYDVQQDEwxFb3VyaXR5IFNlY3VyaXR5IFJlc2VhcmNoIEdyb3Vw  
ZXQgU2VjdXJpdHkgUmVzZWZyY2ggR3JvdXAFTATBgNVBAMTDElUkcGUm9vdCBY  
MTCCAiIwDQYJKoZIhvcNAQEBBQADggIPADCCAggIBAK3oJHP0FDfzm54rVygch  
77ct984kIxuPOZXoHj3dcKi/vVqbvYATyjb3miGbESTtrFj/RQSa78f0uoxmyF+  
0TM8ukj13Xnfs7j/EvEhmkvBioZxaUpmZmyPfjxwv60pIgbz5MDmgK7iS4+3mX6U  
A5/TR5d8mUgjU+g4rk8Kb4Mu0U1XjIB0ttov0DiNewNwIRt18jA8+o+u3dpjq+sW  
T8KOEut+zwvo/7V3LvSye0rgTBIIDHCNAymg4VMk7BPZ7hm/ELNKjD+Jo2FR3qyH  
B5T0Y3HsLuJvW5iB4Y1cNHlsdu87kGJ55tukmi8mxdAQ4Q7e2RCOFvu396j3x+UC  
B5iPNgiV5+I3lg02dZ77DnKxHZu8A/lJBdiB3QW0KtZB6awBdpUKD9jf1b0SHzUv  
KBds0pjBqAlkd25HN7rOrFleaJ1/ctaJxQZBKT5ZPt0m9STJEadao0xAH0ahmbWn  
OlFuhjuefXKnEgV4We0+UXgVCwOPjdAvBbI+e0ocS3MFEVzG6uBQE3xDk3SzynTn  
jh8BCNAw1FtxNrQHusEwMFxIt4I7mKZ9YIqioymCzLq9gwQbooMDQaHwBFebwrbw  
qHyG00aoSCqI3Haadr8faqU9GY/rOPNk3sgrDQoo//fb4hVC1CLQJ13hef4Y53CI  
rU7m2Ys6xt0nUW7/vGT1M0NPAgMBAAGjQjBAMA4GA1UdDwEB/wQEAwIBBjAPBgNV  
HRMBAf8EBTADAQH/MB0GA1UdDgQWBBR5tFnme7b15AFzgAiIyBpY9umbbjANBgkq  
hkiG9w0BAQsFAAOCAgEAVR9YqbyyqFDQDLHYGmkGjYkIrGF1XIpu+ILlaS/V9lZL  
ubhzeFNTIZd+50xx+7LSYK05qAvqyFWhfFQDlnrzuBZ6brJFe+GnY+EgPbk6ZGQ  
3BebYhtF8GaV0nxvuw077x/Py9auJ/GpsMiu/X1+mvoiB0v/2X/qkSsisRcOj/KK  
NftY2PwByVS5UcbMioGziUwthDyC3+6WVwW6LLv3xLfHTjuCvJHIInNzktHCgKQ5  
ORAZI4JMPJ+GslWYHb4phowim57iazTX0oJwTdwJx4nLCgdNb0hdjsnvzqvHu7Ur  
TkXWStAmz0VvyghqPZXjFaH3p03JLF+1/+sKAIuvtd7u+Nxe5AW0wdeRlN8NwdC  
jNPElPzVmbUq4JUagEiuTDkHszxHpFKVK7q4+63SM1N95R1NbdWhscdCb+ZAJzVc  
oyi3B43njTQ05yOf+1CceWxG1bQVs5ZufpsMlj4Ui0/1lvh+wjChP4kqK0J2qxq  
4RgqsahDYVvT9w7jXbyLeiNdd8XM2w9U/t7y0Ff/9yi0GE44Za4rF2LN9d11TPA  
mRGunUHBcnWEvgJBQl9nJEiU0Zsnvgc/ubhPgXRR4Xq37Z0j4r7g1SgEEzwxAS7d  
emyPxgcYxn/eR44/KJ4EBs+1VDR3veyJm+kXQ99b21/+jh5Xos1AnX5iItreGCC=  
-----END CERTIFICATE-----  
)EOF";
```

```
void setup_wifi() {  
    delay(10);  
    Serial.println();  
    Serial.print("Connecting to ");  
    Serial.println(ssid);  
  
    WiFi.mode(WIFI_STA);  
    WiFi.begin(ssid, password);  
  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
  
    Serial.println("");  
    Serial.println("WiFi connected");  
    Serial.println("IP address: ");  
    Serial.println(WiFi.localIP());  
}
```

```

void callback(char* topic, byte* message, unsigned int length) {
    Serial.print("Message arrived on topic: ");
    Serial.print(topic);
    Serial.print(". Message: ");

    int messageValue = -1;
    for (int i = 0; i < length; i++) {
        messageValue = (int)(message[i] - '0'); // Convert char to int
    }
    Serial.println(messageValue);

    if (messageValue == 1) {
        digitalWrite(ledPin, HIGH);
        Serial.println("LED is ON");
        client.publish(mqttPublishTopic, "ON"); // Publish "ON" message
    } else if (messageValue == 0) {
        digitalWrite(ledPin, LOW);
        Serial.println("LED is OFF");
        client.publish(mqttPublishTopic, "OFF"); // Publish "OFF" message
    } else {
        Serial.println("Invalid message received");
    }
}

void connectToMQTT() {
    while (!client.connected()) {
        Serial.println("Connecting to MQTT...");
        if (client.connect("ESP32Client", mqtt_username, mqtt_password)) {
            Serial.println("Connected to MQTT broker");
            client.subscribe(mqttSubscribeTopic);
        } else {
            Serial.print("Failed to connect. Trying again in 5 seconds.");
            delay(5000);
        }
    }
}

void setup() {
    delay(500);
    Serial.begin(9600);
    delay(500);

    pinMode(ledPin, OUTPUT);

    setup_wifi();

    espClient.setCACert(root_ca);
    client.setServer(mqtt_server, mqtt_port);
    client.setCallback(callback);
}

void loop() {
    if (!client.connected()) {
        connectToMQTT();
    }
    client.loop();
}

```

iv. **Menyalakan LED secara remote.**

- Klik **Web Client** pada navbar.



- Selanjutnya isi username dan password seperti yang dibuat sebelumnya kemudian klik **Connect**, atau bisa juga langsung menggunakan **Connect with autogenerated credentials**.
- Pada Topic Subscriptions, isi topik dengan `esp32/led/status` (sesuai variable `mqttPublishTopic`) dan klik **Subscribe**. Sedangkan pada Messages, kita akan melakukan publish message ke topik `esp32/led/control` (sesuai variable `mqttSubscribeTopic`).

Username * Password *

esp32 *****

Disconnect

The WebClient is connected

Topic Subscriptions 0

Subscribe to topics to receive messages from the HiveMQ cluster. You can also set the Quality of Service (QoS) for each topic. The higher the QoS, the more reliable the message delivery is. You can always subscribe to the (#) wildcard to receive all messages.

TOPIC	QOS	ACTIONS
esp32/led/status		Subscribe

Unsubscribe from all topics

Messages 0

Send and see messages that are published to the topics you are subscribed to. If you cannot see any messages, make sure you are subscribed to the correct topics. You can always subscribe to the (#) wildcard to receive all messages.

MESSAGE	TOPIC	QOS	ACTIONS
Your message	esp32/led/control	QoS: v	Send Message

Clear all messages

- Pastikan mikrokontroler Anda sudah berhasil tersambung ke MQTT broker.

```
WiFi connected
IP address:
192.168.0.16
Connecting to MQTT...
Connected to MQTT broker
```

- Selanjutnya pada kolom *Your Message*, isi dengan 1 untuk menghidupkan LED dan 0 untuk mematikan LED kemudian klik **Send Message**. Misalnya kita mengirimkan 1, maka LED akan menyala (jika memang rangkaiannya benar).

```
Message arrived on topic: esp32/led/control. Message: 1
LED is ON
```

- Pada bagian ini, merupakan pesan yang kita dapat dari mikrokontroler dari topik *esp32/led/status*.

Messages 1

Send and see messages that are published to the topics you are subscribed to. If you cannot see any messages, make sure you are subscribed to the correct topics. You can always subscribe to the (#) wildcard to receive all messages.

MESSAGE	TOPIC	QOS	ACTIONS
1	esp32/led/control	QoS: 0	Send Message
ON	esp32/led/status	0	

Cobalah mengirimkan 0, apa yang terjadi? Selamat Anda telah belajar pengiriman-penerimaan data dari dan ke ESP32 dengan MQTT.

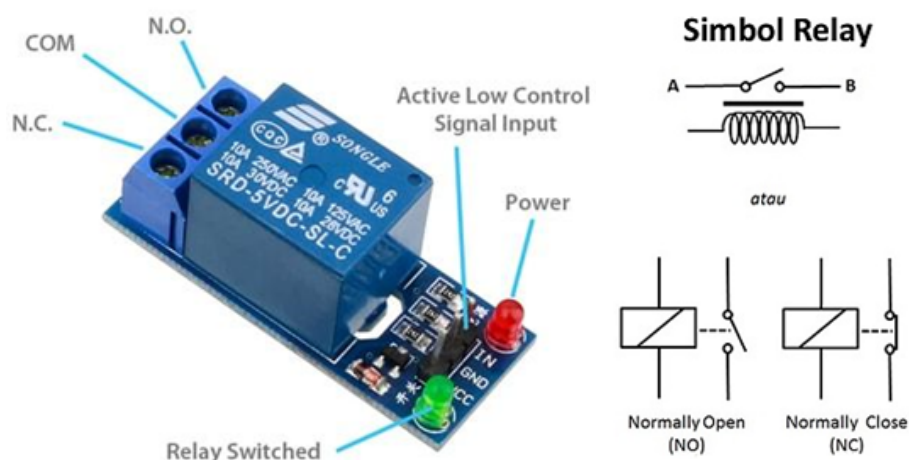
Anda bisa mengimplementasikan ini lebih jauh, seperti mengganti Web Client pada HiveMQ dengan web buatan sendiri misalnya menggunakan bahasa pemrograman [PHP](#).

BAB 5: INTERNET OF THINGS (IOT)

1. Relay

Relay adalah saklar elektromekanikal yang digunakan untuk membuka dan menutup rangkaian listrik serta menstimulasikan listrik kecil menjadi arus yang lebih besar. Pada dasarnya cara kerja relay adalah memutus dan menyambung aliran listrik dalam rangkaian. Bisa dibilang fungsi modul relay sebagai saklar otomatis. Selain digunakan pada rangkaian project Arduino, modul relay 5V juga bisa ditemukan pada jenis kendaraan seperti motor maupun mobil. Kebanyakan, relay 5 volt DC digunakan untuk membuat project yang salah satu komponennya butuh tegangan tinggi atau yang sifatnya AC. Sedangkan kegunaan relay secara lebih spesifik adalah sebagai berikut:

- Menjalankan fungsi logika dari mikrokontroler Arduino
- Sarana untuk mengendalikan tegangan tinggi hanya dengan menggunakan tegangan rendah
- Meminimalkan terjadinya penurunan tegangan
- Memungkinkan penggunaan fungsi penundaan waktu atau fungsi time delay function
- Melindungi komponen lainnya dari kelebihan tegangan penyebab korsleting.
- Menyederhanakan rangkaian agar lebih ringkas.



Berdasarkan gambar skematik relay di atas, berikut ini adalah keterangan dari ketiga pin yang perlu diketahui:

- **COM (Common):** pin yang wajib dihubungkan pada salah satu dari dua ujung kabel yang hendak digunakan.
- **NO (Normally Open):** pin tempat menghubungkan kabel yang satunya lagi bila menginginkan kondisi posisi awal yang terbuka atau arus listrik terputus.
- **NC (Normally Close):** pin tempat menghubungkan kabel yang satunya lagi bila menginginkan kondisi posisi awal yang tertutup atau arus listrik tersambung.

```
1 const int RELAY_PIN = 16;  
2  
3 void setup() {  
4   pinMode(RELAY_PIN, OUTPUT);  
5 }  
6  
7 void loop() {  
8   digitalWrite(RELAY_PIN, HIGH);  
9   delay(100);  
10  digitalWrite(RELAY_PIN, LOW);  
11  delay(100);  
12 }
```

Contoh program untuk memberikan perintah agar switch dalam relay aktif atau tidak aktif pada pin digital 16.

2. Blynk



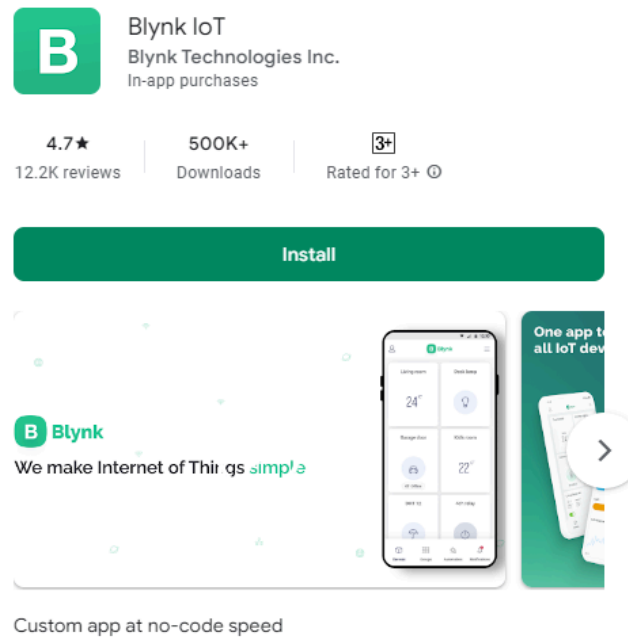
Blynk adalah platform untuk aplikasi OS Mobile (iOS dan Android) yang dapat digunakan untuk kendali module Arduino, Raspberry Pi, ESP8266, WEMOS D1, dan module sejenisnya melalui Internet.

Aplikasi Blynk merupakan wadah kreatifitas untuk membuat antarmuka grafis pada proyek yang akan diimplementasikan hanya dengan metode drag and drop widget. Penggunaannya sangat mudah untuk mengatur semuanya dan dapat dikerjakan dalam waktu kurang dari 5 menit. Blynk tidak terikat pada board atau module tertentu. Dari platform aplikasi inilah kita dapat mengontrol apapun dari jarak

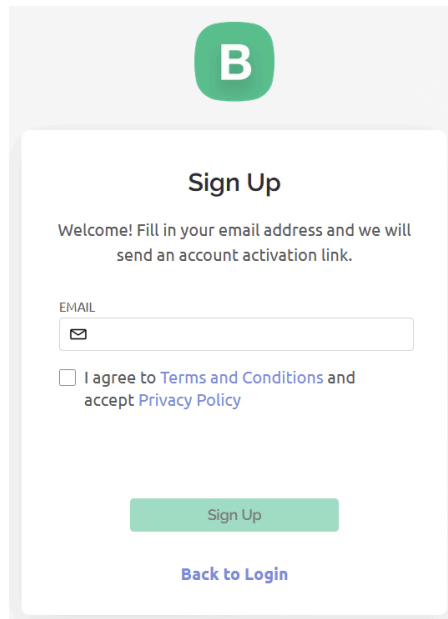
jauh, dimanapun kita berada dan kapanpun. Dengan catatan terhubung dengan internet dengan koneksi yang stabil yang secara umum dinamakan dengan sistem Internet of Things (IoT).

Cara menggunakan aplikasi Blynk :

1. Install aplikasi Blynk IoT di smartphone Anda melalui Playstore atau Appstore.

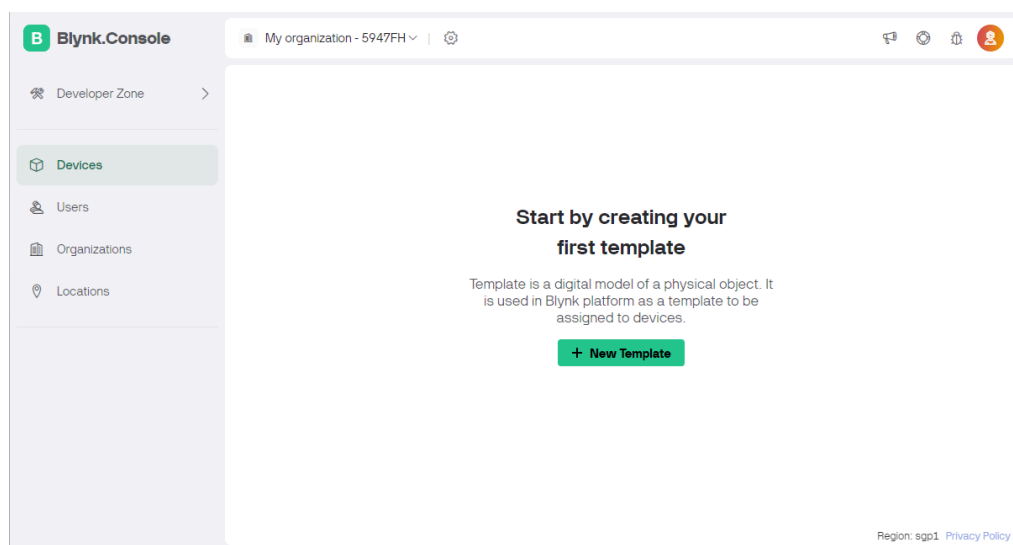


2. Sign Up pada dashboard web Blynk <https://blynk.cloud/dashboard/register>. Buat akun baru yang nantinya akan digunakan untuk membuat dan menyimpan proyek anda. Setelah melakukan registrasi silahkan cek email untuk aktivasi sekaligus membuat password login ke Blynk Console.

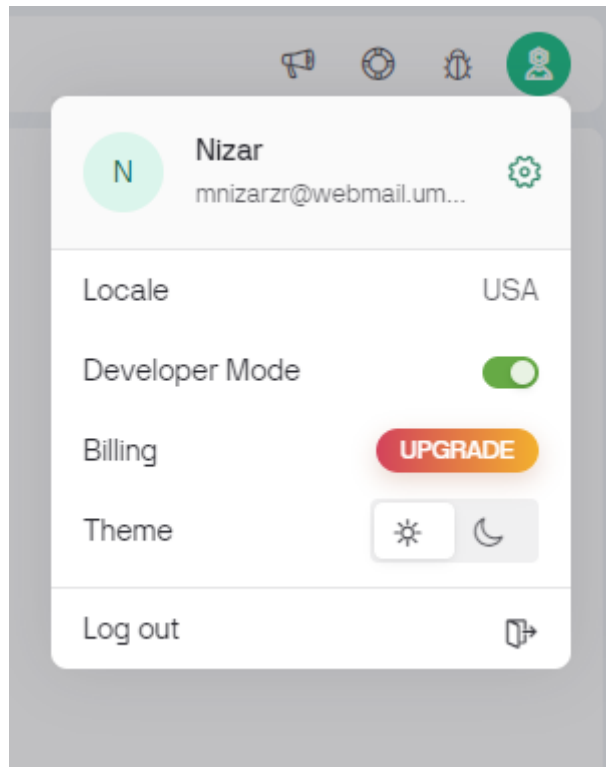


The image shows a 'Sign Up' form for Blynk Console. At the top is the Blynk logo (a green circle with a white 'B'). Below it, the title 'Sign Up' is centered. A welcome message reads: 'Welcome! Fill in your email address and we will send an account activation link.' There is an 'EMAIL' label above a text input field that contains an email icon. Below the input field is a checkbox with the text 'I agree to Terms and Conditions and accept Privacy Policy'. At the bottom of the form are two buttons: a green 'Sign Up' button and a blue 'Back to Login' link.

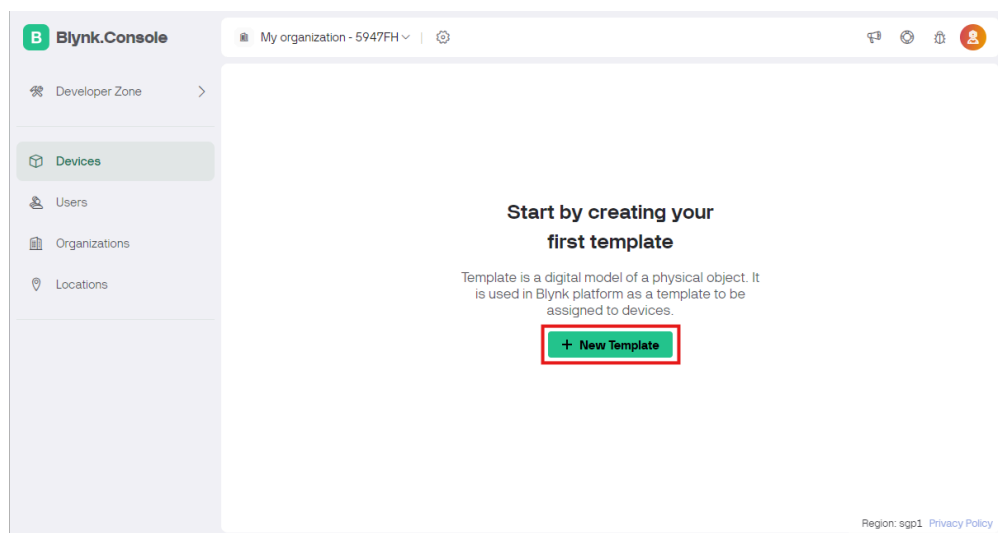
3. Jika sudah bisa login, tampilan website akan diarahkan untuk masuk ke bagian dashboard dari Blynk Console.



4. Aktifkan fitur developer pada submenu user profile.



5. Buatlah template project baru yang akan kalian kembangkan. Pada sub menu Templates klik New Template.



6. Selanjutnya akan muncul bagian pengaturan tipe mikrokontroler dan juga nama aplikasi yang akan dikembangkan. Masukkan nama project kalian dan samakan dengan gambar di bawah

Create New Template

NAME

ESP32 Praktikum15 / 50

HARDWARE

ESP32

CONNECTION TYPE

WiFi

DESCRIPTION

Description0 / 128

Cancel

Done

Pengaturan fitur Aplikasi pada Blynk Console:

- Setelah berhasil membuat sebuah template aplikasi baru, maka pada tampilan selanjutnya akan diarahkan ke dashboard dari proses *editing* template aplikasi. Ada beberapa informasi yang bisa diubah pada kolom sebelah kiri.

×

ESP32 PRAKTIKUM

Home

Datastreams

Web Dashboard

Metadata

Connection Lifecycle

Events & Notifications

User Guides

Mobile Dashboard

ESP32 Praktikum

...

Cancel

Save

Home

What's next?

- ☐ [Configure template](#)
- ☐ [Set Up Datastreams](#)
- ☐ [Set up the Web Dashboard](#)
- ☐ [Add first Device](#)

Template settings

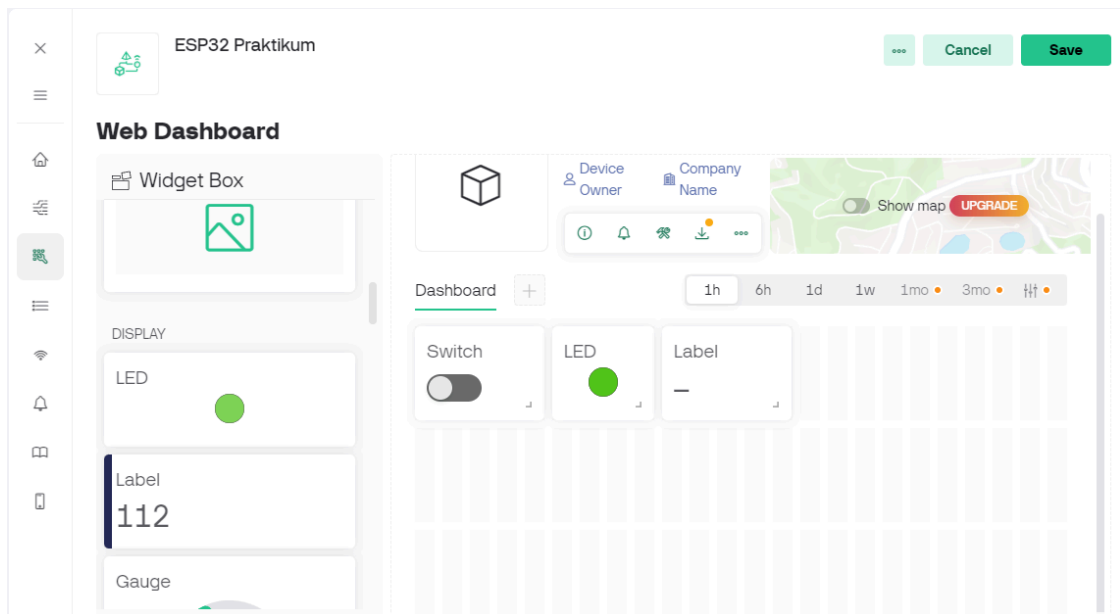
ESP32, WiFi

Firmware configuration

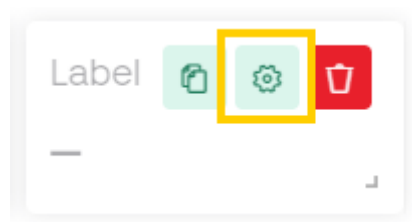
Template ID and Template Name should be declared at the very top of the firmware code.

```
#define BLYNK_TEMPLATE_ID
"TMPL6Y3nDYJ7H"
#define BLYNK_TEMPLATE_NAME "ESP32
Praktikum"
```

- Langkah selanjutnya adalah mendesain tampilan aplikasi pada menu *Web Dashboard* dengan *drag & drop widget* yang tersedia.



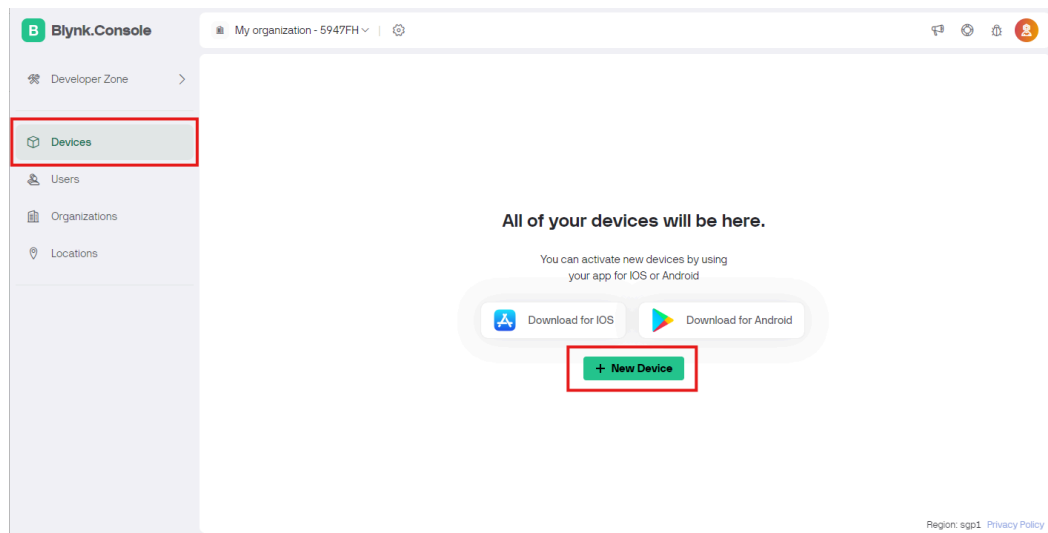
- Untuk melakukan pengaturan pada masing-masing widget, letakkan kursor di atas widget lalu klik ikon settings  .



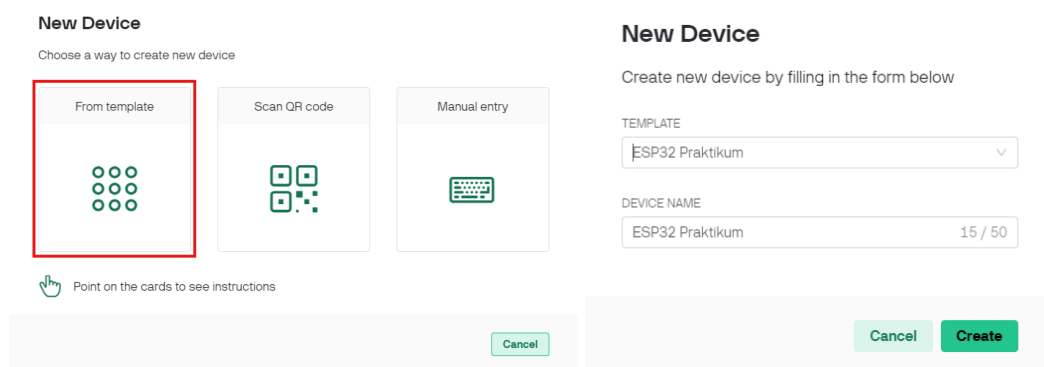
- Jika telah selesai, klik tombol **Save** pada pojok kanan atas untuk menyimpan hasil konfigurasi template aplikasi.

Aktivasi Device untuk Akses Aplikasi :

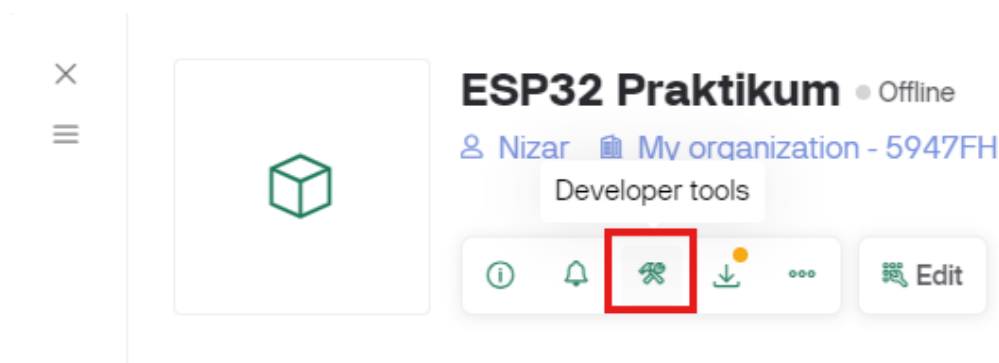
- Setelah membuat template proyek kemudian Anda perlu setting device agar mikrokontroler dan smartphone Anda dapat mengakses template aplikasi yang telah dikonfigurasi sebelumnya. Klik menu **Devices** pada bagian kiri dashboard, kemudian klik **New Device**.



2. Saat memilih pilihan **New Device** pilih **From template** untuk mengambil format aplikasi sesuai dengan desain template yang disusun sebelumnya.



3. Setelah mengklik tombol **Create**, akan diarahkan ke halaman device yang telah dibuat dan muncul popup info mengenai Firmware Configuration pada pojok kanan atas, apabila tertutup Anda bisa mengaksesnya pada Developer Tools. Salin firmware configuration ini untuk digunakan pada mikrokontroler.





4. Silahkan akses aplikasi Blynk IoT pada Hp kalian dan login menggunakan informasi akun Blynk Cloud yang sudah ditambahkan saat awal registrasi.
5. Secara otomatis, tampilan dashboard Blynk App akan menampilkan aplikasi yang telah ter-generate pada Blynk Cloud.
6. Buka dan anda perlu mengonfigurasi **Mobile Dashboard** untuk menampilkan widget pada aplikasi mobile.